스트림 생성 및 관리 과정 문서화

개요

이 문서는 Rust libp2p의 멀티플렉싱 계층에서 스트림이 어떻게 생성되고 관리되는지에 대한 상세한 분석을 제공합니다. Yamux와 Mplex 두 구현체의 스트림 생명주기, 상태 관리, 그리고 리소스 할당 메커니즘을 포함합니다.

스트림 생명주기 개요

기본 스트림 상태

```
┌──────────────┐    Open Frame    ┌──────────────┐
│    Closed    │ ────────────────→ │     Open     │
│              │                   │              │
└──────────────┘                   └──────────────┘
                                          │
                                          │ Data Transfer
                                          ▼
                                   ┌──────────────┐
                                   │    Active    │
                                   │              │
                                   └──────────────┘
                                          │
```

```
                    | Close/Reset
                    ▼
              ┌──────────┐
              |  Closed  |
              |          |
              └──────────┘
```

Yamux 스트림 생성 및 관리

1. 스트림 생성 과정

아웃바운드 스트림 생성

```
fn poll_outbound(
mut self: Pin<&mut Self>,
cx: &mut Context<'_>,
) -> Poll<Result<Self::Substream, Self::Error>> {
let stream = match self.connection.as_mut() {
Either::Left(c) => ready!(c.poll_new_outbound(cx))
.map_err(|e| Error(Either::Left(e)))
.map(|s| Stream(Either::Left(s))),
Either::Right(c) => ready!(c.poll_new_outbound(cx))
.map_err(|e| Error(Either::Right(e)))
.map(|s| Stream(Either::Right(s))),
```

```
}?;
Poll::Ready(Ok(stream))
}
```

과정:

1. **스트림 ID 할당**: 클라이언트는 홀수, 서버는 짝수 ID 사용
2. **yamux 연결에서 새 아웃바운드 스트림 요청**: `poll_new_outbound` 호출
3. **스트림 객체 생성**: 성공 시 `Stream` 래퍼 객체 반환
4. **에러 처리**: 연결 오류 시 적절한 에러 타입으로 변환

인바운드 스트림 처리

```
fn poll_inbound(
mut self: Pin<&mut Self>,
cx: &mut Context<'_>,
) -> Poll<Result<Self::Substream, Self::Error>> {
// 1. 버퍼에서 기존 스트림 확인
if let Some(stream) = self.inbound_stream_buffer.pop_front() {
return Poll::Ready(Ok(stream));
}

// 2. 새로운 스트림 폴링
if let Poll::Ready(res) = self.poll_inner(cx) {
return Poll::Ready(res);
```

```
}

// 3. Waker 등록 후 Pending 반환
self.inbound_stream_waker = Some(cx.waker().clone());
Poll::Pending
}
```

과정:

1. **버퍼 확인**: 기존에 버퍼링된 인바운드 스트림이 있는지 확인
2. **새 스트림 폴링**: 버퍼가 비어있으면 새로운 인바운드 스트림 대기
3. **Waker 등록**: 새 스트림이 없으면 현재 태스크의 Waker 저장
4. **비동기 대기**: `Poll::Pending` 반환하여 다른 태스크 실행 허용

2. 스트림 버퍼링 메커니즘

인바운드 스트림 버퍼

```
/// 스트림을 리셋하기 전에 버퍼링할 스트림 수
/// rust-yamux의 ACK BACKLOG와 동일
const MAX_BUFFERED_INBOUND_STREAMS: usize = 256;

pub struct Muxer<C> {
/// 백프레셔 상황에서 인바운드 스트림을 임시 버퍼링
inbound_stream_buffer: VecDeque<Stream>,
/// 새로운 인바운드 스트림이 사용 가능할 때 호출할 Waker
```

```
inbound_stream_waker: Option<Waker>,
}
```

버퍼링 전략:

- **최대 256개 스트림 버퍼링**: ACK BACKLOG와 동일한 크기
- **FIFO 순서**: `VecDeque` 를 사용하여 선입선출 방식
- **오버플로우 처리**: 버퍼 초과 시 새로운 스트림 드롭
- **Waker 기반 알림**: 효율적인 비동기 처리

백프레셔 처리

```
fn poll(
self: Pin<&mut  Self>,
cx: &mut Context<'_>,
) -> Poll<Result<StreamMuxerEvent, Self::Error>> {
let this = self.get_mut();
let inbound_stream = ready!(this.poll_inner(cx))?;

if this.inbound_stream_buffer.len() >= MAX_BUFFERED_INBOUND_STREAMS {
tracing::warn!(
stream=%inbound_stream.0,
"dropping stream because buffer is full"
);
drop(inbound_stream);
} else {
```

```
this.inbound_stream_buffer.push_back(inbound_stream);
if  let Some(waker) = this.inbound_stream_waker.take() {
waker.wake()
}
}

cx.waker().wake_by_ref();
Poll::Pending
}
```

백프레셔 메커니즘:

1. **버퍼 크기 확인**: 현재 버퍼 크기가 최대치에 도달했는지 확인
2. **스트림 드롭**: 버퍼 초과 시 새로운 스트림을 자동으로 드롭
3. **Waker 알림**: 버퍼에 공간이 있으면 대기 중인 태스크 깨우기
4. **즉시 재스케줄링**: 다른 코드 실행을 위해 현재 태스크 재스케줄링

3. 스트림 I/O 작업

읽기 작업

```
impl AsyncRead for Stream {
fn poll_read(
mut  self: Pin<&mut  Self>,
cx: &mut Context<'_>,
buf: &mut [u8],
```

```
) -> Poll<io::Result<usize>> {
either::for_both!(self.0.as_mut(), s => Pin::new(s).poll_read(cx, buf))
}
}
```

쓰기 작업

```
impl AsyncWrite for Stream {
fn poll_write(
mut self: Pin<&mut Self>,
cx: &mut Context<'_>,
buf: &[u8],
) -> Poll<io::Result<usize>> {
either::for_both!(self.0.as_mut(), s => Pin::new(s).poll_write(cx, buf))
}
}
```

특징:

- **버전 호환성**: `Either` 타입으로 yamux 0.12와 0.13 동시 지원
- **제로 카피**: `either::for_both!` 매크로로 불필요한 복사 방지
- **Pin 안전성**: `Pin<&mut Self>` 사용으로 메모리 안전성 보장

Mplex 스트림 생성 및 관리

1. 스트림 생성 과정

아웃바운드 스트림 생성

```
pub(crate) fn poll_open_stream(&mut  self, cx: &Context<'_>) -> Poll<io::Result<

self.guard_open()?;


// 스트림 제한 확인

if  self.substreams.len() >= self.config.max_substreams {

self.notifier_open.register(cx.waker());

return Poll::Pending;

}


// Open 프레임 전송

let waker = NotifierWrite::register(&self.notifier_write, cx.waker());

match ready!(self.io.poll_ready_unpin(&mut Context::from_waker(&waker))) {

Ok(()) => {

let stream_id = self.next_outbound_stream_id();

let frame = Frame::Open { stream_id };

match  self.io.start_send_unpin(frame) {

Ok(()) => {

self.substreams.insert(

stream_id,

SubstreamState::Open {
```

```
buf: Default::default(),
},
);
self.pending_flush_open.insert(stream_id);
Poll::Ready(Ok(stream_id))
}
Err(e) => Poll::Ready(self.on_error(e)),
}
}
Err(e) => Poll::Ready(self.on_error(e)),
}
}
```

과정:

1. **연결 상태 확인**: `guard_open()` 으로 연결이 열려있는지 확인
2. **스트림 제한 검사**: 최대 스트림 수 제한 확인
3. **스트림 ID 생성**: `next_outbound_stream_id()` 호출
4. **Open 프레임 전송**: 원격 피어에게 새 스트림 생성 요청
5. **상태 등록**: 스트림 상태를 `SubstreamState::Open` 으로 설정
6. **플러시 예약**: `pending_flush_open` 에 추가하여 나중에 플러시

인바운드 스트림 처리

```
pub(crate) fn poll_next_stream(&mut  self, cx: &Context<'_>) -> Poll<io::Result<
```

```
self.guard_open()?;


// 버퍼에서 먼저 읽기 시도
if  let Some(stream_id) = self.open_buffer.pop_back() {
return Poll::Ready(Ok(stream_id));
}


let  mut num_buffered = 0;
loop {
// 버퍼가 가득 찬 경우 양보
if num_buffered == self.config.max_buffer_len {
cx.waker().wake_by_ref();
return Poll::Pending;
}


// 다음 인바운드 Open 프레임 대기
match ready!(self.poll_read_frame(cx, None))? {
Frame::Open { stream_id } => {
if  let Some(id) = self.on_open(stream_id)? {
return Poll::Ready(Ok(id));
}
}
Frame::Data { stream_id, data } => {
```

```
self.buffer(stream_id.into_local(), data)?;

num_buffered += 1;

}

Frame::Close { stream_id } => {

self.on_close(stream_id.into_local());

}

Frame::Reset { stream_id } => self.on_reset(stream_id.into_local()),

}

}

}
```

과정:

1. **버퍼 확인**: `open_buffer` 에서 기존 스트림 확인
2. **프레임 읽기**: 다양한 타입의 프레임 처리
3. **스트림 생성**: `Open` 프레임 수신 시 새 스트림 생성
4. **데이터 버퍼링**: 다른 스트림의 데이터 프레임 버퍼링
5. **양보 메커니즘**: 버퍼가 가득 찬 경우 다른 태스크에 양보

2. 스트림 상태 관리

스트림 상태 정의

```
enum SubstreamState {

/// 스트림이 열려있고 양방향 통신 가능

Open { buf: SmallVec<[Bytes; 4]> },
```

```
/// 로컬에서 Close 프레임을 보냄 (반쪽 닫힘)
SendClosed { buf: SmallVec<[Bytes; 4]> },
/// 원격에서 Close 프레임을 받음 (반쪽 닫힘)
RecvClosed { buf: SmallVec<[Bytes; 4]> },
/// 완전히 닫힘
Closed { buf: SmallVec<[Bytes; 4]> },
/// 리셋됨
Reset { buf: SmallVec<[Bytes; 4]> },
}
```

상태 전환 다이어그램

```
Open Frame
┌──────────────────┐
│       Open       │ ──┐
│                  │   │ Local Close
└──────────────────┘   │
          │            ▼
          │     ┌──────────────────┐
          │     │    SendClosed    │ ──┐
          │     │                  │   │ Remote Close
          │     └──────────────────┘   │
          │                            │
```

```
         | Remote Close             |

         ▼                          ▼

┌─────────────────┐            ┌─────────────────┐

|    RecvClosed   | ──────────→ |      Closed     |

|                 |Local Close|                 |

└─────────────────┘            └─────────────────┘

         |                              ▲

         | Reset                        |

         ▼                              | Reset

┌─────────────────┐                     |

|      Reset      |   ──────────────────┘

|                 |

└─────────────────┘
```

3. 데이터 버퍼링 및 흐름 제어

스트림별 데이터 버퍼

```rust
impl SubstreamState {

fn recv_buf(&mut  self) -> &mut SmallVec<[Bytes; 4]> {

match  self {

SubstreamState::Open { buf } => buf,

SubstreamState::SendClosed { buf } => buf,

SubstreamState::RecvClosed { buf } => buf,
```

```
SubstreamState::Closed { buf } => buf,
SubstreamState::Reset { buf } => buf,
}
}
}
```

특징:

- **SmallVec 최적화**: 작은 버퍼는 스택에, 큰 버퍼는 힙에 할당
- **상태별 버퍼**: 모든 상태에서 버퍼 유지 (데이터 손실 방지)
- **메모리 효율성**: `shrink_to_fit()` 호출로 불필요한 메모리 해제

백프레셔 메커니즘

```
fn buffer(&mut self, id: LocalStreamId, data: Bytes) -> io::Result<()> {
if let Some(state) = self.substreams.get_mut(&id) {
let buf = state.recv_buf();
if buf.len() >= self.config.max_buffer_len {
match self.config.max_buffer_behaviour {
MaxBufferBehaviour::Block => {
self.blocking_stream = Some(id);
return Ok(());
}
MaxBufferBehaviour::ResetStream => {
self.pending_frames.push_front(Frame::Reset { stream_id: id });
self.substreams.insert(id, SubstreamState::Reset {
```

```
buf: Default::default()
});
return Ok(());
}
}
}
buf.push(data);
self.notifier_read.wake_read_stream(id);
}
Ok(())
}
```

백프레셔 전략:

1. **Block 모드**: 버퍼 오버플로우 시 전체 연결 차단
2. **ResetStream 모드**: 버퍼 오버플로우 시 해당 스트림 리셋
3. **알림 시스템**: 데이터 버퍼링 후 관련 태스크 깨우기

4. 스트림 I/O 작업

읽기 작업

```
impl<C> AsyncRead for Substream<C>
where
C: AsyncRead + AsyncWrite + Unpin,
{
```

```rust
fn poll_read(
self: Pin<&mut  Self>,
cx: &mut Context<'_>,
buf: &mut [u8],
) -> Poll<io::Result<usize>> {
let this = self.get_mut();


loop {
// 현재 프레임에서 데이터 읽기 시도
if !this.current_data.is_empty() {
let len = cmp::min(this.current_data.len(), buf.len());
buf[..len].copy_from_slice(&this.current_data.split_to(len));
return Poll::Ready(Ok(len));
}


// 다음 데이터 프레임 읽기
match ready!(this.io.lock().poll_read_stream(cx, this.id))? {
Some(data) => {
this.current_data = data;
}
None => return Poll::Ready(Ok(0)),
}
}
```

```
}

}
```

특징:

- **프레임 기반 처리**: 데이터를 프레임 단위로 처리
- **부분 읽기 지원**: 큰 프레임을 여러 번에 나누어 읽기 가능
- **EOF 처리**: 스트림 종료 시 0 바이트 반환

쓰기 작업

```
pub(crate) fn poll_write_stream(

&mut  self,

cx: &Context<'_>,

id: LocalStreamId,

buf: &[u8],

) -> Poll<io::Result<usize>> {

self.guard_open()?;


// 스트림이 쓰기 가능한지 확인

match  self.substreams.get(&id) {

None | Some(SubstreamState::Reset { .. }) => {

return Poll::Ready(Err(io::ErrorKind::BrokenPipe.into()))

}

Some(SubstreamState::SendClosed { .. }) | Some(SubstreamState::Closed { .. }) =>
```

```
return Poll::Ready(Err(io::ErrorKind::WriteZero.into()))
}
Some(SubstreamState::Open { .. }) | Some(SubstreamState::RecvClosed { .. }) => {
// 스트림이 쓰기 가능. 계속 진행.
}
}

// 전송할 프레임 크기 결정
let frame_len = cmp::min(buf.len(), self.config.split_send_size);

// 데이터 프레임 전송
ready!(self.poll_send_frame(cx, || {
let data = Bytes::copy_from_slice(&buf[..frame_len]);
Frame::Data {
stream_id: id,
data,
}
}))?;

Poll::Ready(Ok(frame_len))
}
```

특징:

- **상태 기반 검증**: 스트림 상태에 따른 쓰기 가능성 확인
- **프레임 크기 제한**: `split_send_size` 설정에 따른 프레임 분할
- **에러 처리**: 적절한 I/O 에러 타입 반환

스트림 종료 및 정리

1. 정상 종료 (Close)

Yamux 종료

```
fn poll_close(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Result<(
match self.connection.as_mut() {
Either::Left(c) => c.poll_close(cx).map_err(|e| Error(Either::Left(e))),
Either::Right(c) => c.poll_close(cx).map_err(|e| Error(Either::Right(e))),
}
}
```

Mplex 종료

```
pub(crate) fn poll_close_stream(
&mut self,
cx: &Context<'_>,
id: LocalStreamId,
) -> Poll<io::Result<()>> {
match self.substreams.remove(&id) {
```

```
Some(SubstreamState::Open { buf }) => {
if  self.poll_send_frame(cx, || Frame::Close { stream_id: id })?.is_pending() {
self.substreams.insert(id, SubstreamState::Open { buf });
Poll::Pending
} else {
self.substreams.insert(id, SubstreamState::SendClosed { buf });
Poll::Ready(Ok(()))
}
}
// ... 다른 상태 처리
}
}
```

2. 강제 종료 (Reset)

스트림 드롭 처리

```
pub(crate) fn drop_stream(&mut  self, id: LocalStreamId) {
// 읽기 대기 중인 태스크 깨우기
self.notifier_read.wake_read_stream(id);

match  self.substreams.remove(&id) {
Some(SubstreamState::Open { .. }) => {
// Reset 프레임 전송 예약
```

```
self.pending_frames.push_front(Frame::Reset { stream_id: id });
}
Some(SubstreamState::RecvClosed { .. }) => {
// Close 프레임 전송 예약
self.pending_frames.push_front(Frame::Close { stream_id: id });
}
// ... 다른 상태는 추가 프레임 불필요
}
}
```

정리 과정:

1. **대기 태스크 알림**: 해당 스트림을 기다리는 태스크 깨우기
2. **상태별 처리**: 스트림 상태에 따른 적절한 종료 프레임 전송
3. **리소스 해제**: 스트림 관련 모든 리소스 정리
4. **제한 해제**: 스트림 수 제한 아래로 떨어지면 대기 중인 태스크 알림

성능 최적화 기법

1. 메모리 최적화

Yamux

- **VecDeque 사용**: O(1) 삽입/제거 연산
- **Either 타입**: 제로 카피 버전 호환성
- **Waker 재사용**: 불필요한 할당 방지

Mplex

- **SmallVec 최적화**: 작은 버퍼는 스택 할당
- **Bytes 타입**: 참조 카운팅으로 메모리 공유
- **IntMap 사용**: 정수 키에 최적화된 해시맵

2. 동시성 최적화

비동기 처리

- **Poll 기반**: 논블로킹 I/O 연산
- **Waker 시스템**: 효율적인 태스크 스케줄링
- **백프레셔**: 메모리 사용량 제어

배치 처리

- **프레임 배치**: 여러 프레임을 한 번에 전송
- **플러시 지연**: 불필요한 플러시 연산 최소화
- **버퍼 통합**: 작은 데이터 조각들을 큰 프레임으로 통합

3. 에러 처리 최적화

빠른 실패

- **상태 검증**: 잘못된 상태에서 즉시 에러 반환
- **연결 상태 확인**: 닫힌 연결에서 빠른 에러 처리
- **리소스 제한**: 제한 초과 시 즉시 거부

복구 메커니즘

- **스트림 리셋**: 문제가 있는 스트림만 종료
- **연결 유지**: 개별 스트림 오류가 전체 연결에 영향 없음

- **재시도 로직**: 일시적 오류에 대한 자동 재시도

모니터링 및 디버깅

1. 로깅 시스템

```
tracing::debug!(
connection=%self.id,
substream=%stream_id,
total_substreams=%self.substreams.len(),
"New outbound substream"
);
```

```
tracing::warn!(
stream=%inbound_stream.0,
"dropping stream because buffer is full"
);
```

2. 메트릭 수집

- **스트림 수**: 현재 활성 스트림 개수
- **버퍼 사용량**: 각 스트림의 버퍼 크기
- **처리량**: 초당 처리되는 데이터 양
- **지연시간**: 스트림 생성부터 첫 데이터까지의 시간

3. 디버깅 도구

- **상태 덤프**: 현재 모든 스트림의 상태 출력
- **프레임 추적**: 송수신되는 모든 프레임 로깅
- **성능 프로파일링**: 병목 지점 식별

결론

Rust libp2p의 스트림 생성 및 관리 시스템은 다음과 같은 핵심 특징을 가집니다:

주요 설계 원칙

1. **비동기 우선**: 모든 작업이 논블로킹 방식으로 설계
2. **백프레셔 지원**: 메모리 사용량과 처리 속도 제어
3. **상태 기반 관리**: 명확한 스트림 생명주기 정의
4. **에러 격리**: 개별 스트림 오류가 전체 연결에 영향 없음

성능 특성

- **Yamux**: 높은 처리량, 정교한 흐름 제어, 복잡한 구현
- **Mplex**: 낮은 지연시간, 단순한 구현, 제한적인 기능

사용 권장사항

- **고성능 요구사항**: Yamux 선택
- **단순함 우선**: Mplex 선택
- **하이브리드 환경**: 상황에 따른 동적 선택

이러한 설계를 통해 libp2p는 다양한 네트워크 환경과 애플리케이션 요구사항에 대응할 수 있는 유연하고 효율적인 스트림 멀티플렉싱 솔루션을 제공합니다.